Package 'BOMcatchr'

June 5, 2026

Type Package

Title Climate data anywhere in Australia and any time-step

Version 0.2.10

Maintainer Tim Peterson <tim.peterson@monash.edu>

Description NetCDF files of the Bureau of Meteorology Australian Water Availability Project daily national climate grids are built and used for the efficient extraction of point and catchment area weighted precipitation, minimum temperature, maximum temperature, vapour pressure, solar radiation and various measures of evapotranspiration. For details on the source climate data see <http://www.bom.gov.au/jsp/awap/>. Please cite <https://doi.org/10.1002/hyp.13637>.

Depends R (>= 4.0)

Imports archive, Evapotranspiration (>= 1.14), RNetCDF, utils, sf, zoo, methods, xts, stats, curl, RCurl, elevatr, progress, terra (>= 1.9)

Suggests knitr, rmarkdown, testthat (>= 3.0.0),

VignetteBuilder knitr

BugReports https://github.com/peterson-tim-j/BOMcatchr/issues

URL https://github.com/peterson-tim-j/BOMcatchr, https://peterson-tim-j.github.io/BOMcatchr/

License GPL-3

Encoding UTF-8

LazyData true

RoxygenNote 7.3.3

Config/testthat/edition 3

Author Tim Peterson [aut, cre] (ORCID: <https://orcid.org/0000-0002-1885-0826>), Conrad Wasko [aut] (ORCID: <https://orcid.org/0000-0002-9166-8289>)

Contents

`catchments` *Example catchment boundaries.*

Description

Two example catchment boundaries as a SpatialPolygonsDataFrame. The catchments are Creswick Creek (ID 407214, Vic., Australia, see http://www.bom.gov.au/water/hrs/#id=407214) and Bet Bet Creek (ID 407220, Vic., Australia, see http://www.bom.gov.au/water/hrs/#id=407220).

The catchments can be used to extract catchment average climate data usng `extract.data`

Usage

```
catchments()
```

Value

terra::SpatVector

See Also

`extract.data` for extracting catchment average climate data.

`extract.data` *Extract data over catchment area and duration .*

Description

extract.data extracts the AWAP climate data for each point or polygon. For the latter, either the daily spatial mean and variance (or user defined function) of each climate metric is calculated or the spatial data is returned.

Usage

```
extract.data(
  ncdfFilename = file.path(getwd(), "AWAP.nc"),
  extractFrom = as.Date("1900-01-01", "%Y-%m-%d"),
  extractTo = as.Date(Sys.Date(), "%Y-%m-%d"),
  vars = "",
  locations = NA,
  temporal.timestep = "daily",
  temporal.function.name = "mean",
  spatial.function.name = "var",
  interp.method = "",
  missing.method = c("5", "linear", "mean"),
```

```
  ET.function = "ET.MortonCRAE",
  ET.DEM.res = 10,
  ET.Mortons.est = "wet areal ET",
  ET.Turc.humid = F,
  ET.timestep = "monthly",
  ET.missing_method = "DoY average",
  ET.abnormal_method = "DoY average",
  ET.constants = list()
)
```

Arguments

`ncdfFilename`	is a full file name (as string) to the netCDF file.
`extractFrom`	is a date string specifying the start date for data extraction. The default is `"1900-1-1"`.
`extractTo`	is a date string specifying the end date for the data extraction. The default is today's date as YYYY-MM-DD.
`vars`	is a vector of variables names to extract. The available variables are: daily precipitation, daily minimum temperature, daily maximum temperature, daily 3pm vapour pressure grids and daily solar radiation and evapotranspiration. The input vector for these options are `c('tmax', 'tmin', 'precip', 'precip.monthly', 'vprp', 'solarrad', 'et')`. Importantly, the input `et` is calculated from the available gridded data (see `ET.` inputs below). To calculate the ET, all of the required inputs for the calculation ET must also be extracted (i.e. the input for such would generally be `c('tmax', 'tmin', 'precip', 'vprp', 'solarrad', 'et')`. Any or all of the defaults are available. The default `''` and this will result in all of the variables in the netCDF file and provided by `rownames(BOMcatchr::grid.summary(ncdfFilena`
`locations`	is either the full file name to an ESRI shape file of points or polygons (latter assumed to be catchment boundaries) or a shape file already imported using readShapeSpatial(). Either way the shape file must be in long/lat (i.e. not projected), use the ellipsoid GRS 80, and the first column must be a unique ID.
`temporal.timestep`	character string for the time step of the output data. The options are `daily`, `weekly`, `monthly`, `quarterly`, `annual` or a user-defined index for, say, water-years (see `xts::period.apply`). The default is `daily`.
`temporal.function.name`	character string for the function name applied to aggregate the daily data to `temporal.timestep`. Note, NA values are not removed from the aggregation calculation. If this is required then consider writing your own function. The default is `mean`.
`spatial.function.name`	character string for the function name applied to estimate the daily spatial spread in each variable. If `NA` or `""` and `locations` is a polygon, then the spatial data is returned. The default is `var`.
`interp.method`	character string defining the method for interpolating the gridded data (see `terra::extract`). The options are: `'simple'`, `'bilinear'` and `''`. The default is `''`. This will set the interpolation to `'simple'` when `locations` is a polygon(s) and to `'bilinear'` when `locations` are points.
`missing.method`	three character vector for the settings to fill gaps in the source data. The three inputs control the following. # 1) the infilling of small holes in the source grids

using focal(), which takes the mean of the non-NA surrounding grid cells. The user input controls the maximum hole size filled, in units of number of grid cells. Where the hole is greater than the input, a gap within the hole will remain. The default maximum hole size infilled is 5x5 grid cells. 2) Gaps that remain after the hole infilling, or time steps with no observations, are interpolated over time. Only gaps with observations prior to the gap are interpolated. The interpolation method is user-defined and includes `'constant'`, `'linear'`, `'fmm'`, `'periodic'`, `'natural'`, `'monoH.FC'` and `'hyman'`. The default is `'linear'`. See `approx` and `splinefun` for details. 3) Gaps that remain (often due to the extraction date being prior to the start of the observation record of a variable) are estimated from, say, the mean for each day of the year. Specifically, the extracted observed data is allocated to each calender day. If, say, there are ten years of daily data then each day of the year will have ten observations. All gaps of the same corresponding calender day will then be assigned a value from a user-defined function from these observations (NB: when only one observed value exists for the day, then the observed value is returned). The default function is `mean`. Other standard functions (e.g. `median`) or user defined functions can be used. The default for this input is `c('5', 'linear', 'mean')`. All gap filling method can be turned off with the input `c('0', '', '')`, which is useful to identify the interpolated data points.

`ET.function` character string for the evapotranspiration function to be used. The methods that can be derived from the AWAP data are are `ET.Abtew`, `ET.HargreavesSamani`, `ET.JensenHaise`, `ET.Makkink`, `ET.McGuinnessBordne`, `ET.MortonCRAE` , `ET.MortonCRWE`, `ET.Turc`. Default is `ET.MortonCRAE` i.e. the complementary relationship for areal evapotranspiration .

`ET.DEM.res` is the zoom resolution for the land surface elevation and is required to calculate the ET. `elevatr` package is used to extract elevation (metres) from AWS Open Data Terrain Tiles. This input controls the zoom resolution. Higher values increase accuracy, but are significantly slower. See details. Default is 10.

`ET.Mortons.est` character string for the type of Morton's ET estimate. For `ET.MortonCRAE`, the options are `potential ET`,`wet areal ET` or `actual areal ET`. For `ET.MortonCRWE`, the options are `potential ET` or `shallow lake ET`. The default is `wet areal ET`, which when `ET.function = 'ET.MortonCRAE'` it provides an estimate of the wet areal potential evapotranspiration.

`ET.Turc.humid` logical variable for the Turc function using the humid adjustment.See `ET.Turc`. For now this is fixed at `F`.

`ET.timestep` character string for the evapotranpiration time step. Options are `daily`, `monthly`, `annual` but the options are dependent upon the chosen `ET.function`. The default is `'monthly'`.

`ET.missing_method`
character string for interpolation method for missing variables required for ET calculation. The options are `'monthly average'`, `'seasonal average'`, `'DoY average'` and `'neighbouring average'`. Default is `'DoY average'` but when the extraction duration is less than two years, the default is `'neighbouring average'`. See `ReadInputs`

`ET.abnormal_method`
character string for interpolation method for abnormal variables required for ET calculation (e.g. Tmin > Tmax). Options and defaults are as for `ET.missing_method`. See `ReadInputs`

`ET.constants` list of constants from Evapotranspiration package required for ET calculations. To get the data use the command `data(constants)`. Default is `list()`.

Details

Daily data is extracted and can be aggregated to a weekly, monthly, quarterly, annual or a user-defined timestep using a user-defined function (e.g. sum, mean, min, max as defined by `temporal.function.name`). The temporally aggregated data at each grid cell is then used to derive the spatial mean or the spatial variance (or any other function as defined by `spatial.function.name`).

The calculation of the spatial mean uses the fraction of each AWAP grid cell within the catchment polygon. The variance calculation (or user defined function) does not use the fraction of the grid cell and returns NA if there are <2 grid cells in the catchment boundary. Prior to the spatial aggregation, evapotranspiration (ET) can also calculated; after which, say, the mean and variance PET can be calculated.

The data extraction will by default be undertaken from 1/1/1900 to yesterday, even if the netCDF grids were only built for a subset of this time period. If the latter situation applies, it is recommended that the extraction start and end dates are input by the user.

The ET can be calculated using one of eight methods at a user defined calculation time-step; that is the `ET.timestep` defines the time step at which the estimates are derived and differs from the output timestep as defined by `temporal.function.name`). When `ET.timestep` is monthly or annual then the ET estimate is linearly interpolated to a daily time step (using zoo:na.spline()) and then constrained to >=0. In calculating ET, the input data is pre-processed using Evapotranspiration::ReadInputs() such that missing days, missing entries and abnormal values are interpolated (by default) with the former two interpolated using the "DoY average", i.e. replacement with same day-of-the-year average. Additionally, when AWAP solar radiation is required for the ET function, data is only available from 1/1/1990. To derive ET values <1990, the average solar radiation for each day of the year from 1/1/990 to "extractTo" is derived (i.e. 365 values) and then applied to each day prior to 1990. Importantly, in this situation the estimates of ET <1990 are dependent upon the end date extracted. Re-running the estimation of ET with a later extractTo data will change the estimates of ET prior to 1990.

Some measures of ET require land surface elevation. Here, elevation at the centre of each 0.05 degree grid cell is obtained using the `elevatr` package, which here uses data from the Amazon Web Service AWS Open Data Terrain Tiles. The data sources change with the user set `ET.DEM.res` zoom. The options are 1 to 15. The default of 10 is reasonably computationally efficient and has a resolution of about 108 m, with is acceptable given the 0.05 degree resolution of the BOM source data grids equates to about 5 km x 5 km. For details see https://github.com/tilezen/joerd/blob/master/docs/data-sources.md

Also, when "locations" is points (not polygons), then the netCDF grids are interpolate using bilinear interpolation of the closest 4 grid cells.

Lastly, data is extracted for all time points and no temporal infilling is undertaken if the grid cells are blank.

Value

When `locations` are polygons and `spatial.function.name` is not `NA` or `""`, then the returned variable is a list variable containing two data.frames. The first is the areal aggregated climate metrics named `catchmentTemporal.` with a suffix as defined by `temporal.function.name`). The second is the measure of spatial variability named `catchmentSpatial.` with a suffix as defined by `spatial.function.name`).

When `locations` are polygons and `spatial.function.name` does equal `NA` or `""`, then the returned variable is a `terra::vect` object where the first column is the location/catchment IDs and the latter columns are the results for each variable at each time point as defined by `temporal.timestep`.

When `locations` are points, the returned variable is a data.frame containing daily climate data at each point.

See Also

grid.build for building the NetCDF files of daily climate data.

Examples

```
# The example shows how to extract and save data.
#---------------------------------------

# Set dates for building netCDFs and extracting data.
# Note, to reduce runtime this is done only a fortnight (14 days).
startDate = as.Date("2000-01-01","%Y-%m-%d")
endDate = as.Date("2000-01-14","%Y-%m-%d")

# Set names for netCDF file.
ncdfFilename = tempfile(fileext='.nc')

# Build netCDF grids and over a defined time period.
# Only precip data is to be added to the netCDF files.
# This is because the URLs for the other variables are set to zero.

file.name = grid.build(ncdfFilename=ncdfFilename,
             updateFrom=startDate,
             updateTo=endDate,
             vars = c('precip'))

# Load example catchment boundaries and remove all but the first.
# Note, this is done only to speed up the example runtime.
catch = catchments()
catch = catch[1,]

# Extract daily precip. data (not Tmin, Tmax, VPD, ET).
# Note, the input "locations" can also be a file to a ESRI shape file.
climateData = extract.data(ncdfFilename=file.name,
              extractFrom=startDate,
              extractTo=endDate,
              vars = c('precip'),
              locations=catch,
              temporal.timestep = 'daily')

# Extract the daily catchment average data.
climateDataAvg = climateData$catchmentTemporal.mean

# Extract the daily catchment variance data.
climateDataVar = climateData$catchmentSpatial.var

# Remove temp. files
unlink(ncdfFilename)
```

---

grid.ages *Age of netCDF data and updates required.*

Description

grid.ages age of the netCDF date and which time points likely have BoM updates.

Usage

```
grid.ages(
  ncfile,
  data.src = grid.sources(),
  silent = F,
  plot.sourcedate = F,
  today = Sys.Date()
)
```

Arguments

ncfile	file name of the netCDF data file built by the package function grid.build.
data.src	is a data.frame of the source grid information. Default is from grid.sources.
silent	is a logical for silencing console outputs. Default is F.
plot.sourcedate	is a logical for plotting the date that the data for each time point was downloaded. Default is F.
today	is today's date. Default is Sys.Date(). This input is only provided to demonstrate the function within the vignettes.

Details

This function compares the download date of data within an existing netCDF file and compares them against the BoM schedule of data updates (provided (see grid.sources),

Value

list of data.frames, one for each variable in the netCDF file. Each data frame contains the following columns:

- Date: observation date,
- Source.date: date observation was downloaded from the BoM.
- Source.age : age of observation in days.
- 1 < age <= 7 : logical if Source.age is greater than the left threshold and less than or equal to right duration threshold.
- age <= 7 : logical if Source.age is less than or equal to the maximum duration threshold.

See Also

grid.sources for update threshold data.

`grid.build` *Build a netCDF file of climate data.*

Description

`grid.build` builds one netCDF file containing Australian climate data.

Usage

```
grid.build(
  ncdfFilename = file.path(getwd(), "AWAP.nc"),
  updateFrom = as.Date("1900-01-01", "%Y-%m-%d"),
  updateTo = as.Date(Sys.Date() - 2, "%Y-%m-%d"),
  vars = "",
  keepFiles = FALSE,
  compressionLevel = 5,
  vars.sourceData = grid.sources()
)
```

Arguments

`ncdfFilename`	is a file path (as string) and name to the netCDF file. If only a file name is given, then the file is assumed to exist / be created in `getwd()`. The default file name and path is `file.path(getwd(),'AWAP.nc')`.
`updateFrom`	is a date string specifying the start date for the AWAP data. If `ncdfFilename` is specified and exist, then the netCDF grids will be updated with new data from `updateFrom`. To update the file from the end of the last day in the file set `updateFrom=NA`. The default is `"1900-1-1"`.
`updateTo`	is a date string specifying the end date for the AWAP data. If `ncdfFilename` is specified and exist, then the netCDF grids will be updated with new data to `updateFrom`. The default is two days ago as YYYY-MM-DD.
`vars`	is a vector of variables names to build or update. The available variables are: daily precipitation, monthly precipitation, daily minimum temperature, daily maximum temperature, daily 3pm vapour pressure grids and daily solar radiation. Any or all of the defaults are available. If `vars=''` and the netCDF does not exist, then the default is `c('precip', 'precip.monthly','tmin', 'tmax', 'vprp', 'solarrad')` and provided by `rownames(grid.sources())`. However, if `vars=''` and the netCDF file does exist, then default is to use the variable names in the file.
`keepFiles`	is a logical scalar to keep the downloaded AWAP grid files. The default is `FALSE`.
`compressionLevel`	is the netCDF compression level between 1 (low) and 9 (high), and `NA` for no compression. Note, data extraction runtime may slightly increase with the level of compression. The default is `5`.
`vars.sourceData`	is a data.frame of variable unit, time step and source URLs. This input is provided in-case the default URLs need to be changed. The default is `grid.sources()`

Details

grid.build creates one netCDF file of daily climate data.

One netCDF file is created than contains precipitation, minimum daily temperature, maximum daily temperature and vapour pressure and the solar radiation data. It should span from 1/1/1900 to yesterday and requires ~20GB of hard-drive space (using default compression). For the solar radiation, spatial gaps are infilled using a 3x3 moving average repeated 3 times. To minimise the runtime in extracting data, the netCDF file should be stored locally and not on a network drive. Also, building the file requires installation of 7zip.

The climate data is sourced from the Bureau of Meteorology Australian Water Availability Project (http://www.bom.gov.au/jsp/awap/. For details see Jones et al. (2009).

The output from this function is required for all data extraction functions within this package and must be ran prior.

The function can be used to a build netCDF file from scratch or to update an existing netCDF file previously derived from this function. To not build or update a variable, set its respective URL to NA.

Value

A string containing the full file name to the netCDF file.

References

David A. Jones, William Wang and Robert Fawcett, (2009), High-quality spatial climate data-sets for Australia, Australian Meteorological and Oceanographic Journal, 58 , p233-248.

See Also

extract.data for extracting catchment daily average and variance data.

Examples

```
# The example shows how to build the netCDF data cubes.
# For extra example see \url{https://github.com/peterson-tim-j/BOMcatchr/blob/master/README.md}
#---------------------------------------

# Set dates for building netCDFs and extracting data from 15 to 5 days ago.
startDate = as.Date(Sys.Date()-15,"%Y-%m-%d")
endDate = as.Date(Sys.Date()-5,"%Y-%m-%d")

# Set names for the netCDF file (in the system temp. directory).
ncdfFilename = tempfile(fileext='.nc')


# Build netCDF grids for daily precipitation and only over the defined time period.
file.names = grid.build(ncdfFilename=ncdfFilename,
             updateFrom=startDate,
             updateTo=endDate,
             vars = c('precip'))

# Now, to demonstrate updating the netCDF grids to one day ago, rerun with
# the same file names but \code{updateFrom=NA}.
file.names = grid.build(ncdfFilename=ncdfFilename,
             updateFrom=NA)
```

```
# Remove temp. file
unlink(ncdfFilename)
```

`grid.sources` *Source data URLs and attributes.*

Description

`grid.sources` get available variables, units and URLs to BoM gridded data.

Usage

```
grid.sources()
```

Details

This function returns a list of available variables, unit, time step and URLs used to download the meteorological data.

Value

data.frame of the source data location and properties required by the package:

- `label`: string description of the variable,
- `units`: string for units of the variable.
- `time.step` : string for the time step of the data. `days` or `months` are accepted.
- `data.URL` : string of URL to the source gridded data.
- `data.file.extension` : string for the file extension of the downloaded compressed source data.
- `data.file.format` : string for file extension to the file required within the downloaded file.
- `ncdf.name` : string for the name of the variable once input to the package netCDF file.
- `ellipsoid.crs` : string for Coordinate Reference System (CRS) for the gridded data ellipsoid.
- update.days : string for the update schedule of the source data from https://www.bom.gov.au/climate/austmaps/update-schedule.shtml. Each listed item in the string is the number of days after which the BoM update a data grid. That is, when the data for a given date is, say, 7 days old then the BoM may update the data for that day. This field is used to identify netCDF time points requiring update to the latest data.

Examples

```
vars = grid.sources()
```

grid.summary *Summarise existing netCDF file.*

Description

`grid.summary` sumarises the netCDF variables, units and date ranges.

Usage

```
grid.summary(ncfile)
```

Arguments

`ncfile`	file name of the netCDF data file built by this package.

Details

This function opens an existing netCDF file built using the package and returns a data.frame of variables, unit, stand and end dates of the data.

Value

data.frame summarising the attributes of each variable in the netCDF file. The data.frame includes the following columns:

- `group`: string for the netCDF group in which the variable is placed,
- `var.string`: string for the group and variable name.
- `from` : Date variable for the start of the first time step containing data.
- `to` : Date variable for the end of the last time step containing data.
- `time.step` : string for the time step of the data.
- `time.datum` : string for time datum from which netCDF layers are indexed.
- `units` : string for units of the variable.
- `ellipsoid.crs` : string for Coordinate Reference System (CRS) for the gridded data ellipsoid.

raingauge *Example rain gauge data.*

Description

Daily data for rain gauge 63005 from 2010. It used by a vignette to evaluate the extracted point data. The data is from Bathurst, NSW.

Usage

```
data("raingauge")
```

Format

data.frame of 8 columns and 365 rows.

Source

Bureau of Meteorology, Australia. See https://www.bom.gov.au/climate/data/index.shtml

Index